

Designing an Azure Ops Agent with Human Approval and Kubernetes Actions

Overview

This document describes, end to end, how to build an operations agent that:

Proposes an action → asks your permission → if you approve, runs a real command against your Kubernetes cluster and returns the results.

The design assumes:

- Azure AI Foundry (Microsoft Foundry) is used for the **agent brain**.
- A **local or server-side controller** (your code) has access to your clusters and executes `kubectl`/API calls.
- Interaction starts in the **Azure portal** (Agents playground) and then moves to a **local project** that wires tools, approval, and execution.

The document is written for an engineer with DevOps/SRE experience and comfort with Azure, Kubernetes, and Python.

High-Level Architecture

At a high level, the system has four parts:

1. Azure AI Foundry project and agent

- Foundry project hosts model deployments and the agent definition.
- The agent is configured with instructions and tools (function calling / OpenAPI).
- The agent itself *never* has direct access to your cluster; it only emits **tool calls**.

2. Function / OpenAPI tools for the agent

- Tools expose safe, parameterized operations such as `list_pods(cluster, namespace)` or `describe_pod(cluster, namespace, name)`.
- Tools are defined according to Microsoft Foundry's tool catalog and function calling concepts.

3. Ops controller service (your code)

- Authenticates to your clusters (kubeconfig, AKS credentials, etc.).
- Receives tool call requests from the agent, asks you for approval, then executes real commands via `kubectl` or the Kubernetes API.
- Returns sanitized results to the agent as tool outputs.

4. Client UI on your desktop

- CLI, TUI, VS Code extension, or simple web UI that:
 - Shows the agent conversation.
 - Surfaces "agent wants to run X" prompts.
 - Lets you approve/deny.
- Uses the Azure AI Agents/Projects SDK or REST endpoints to talk to the agent.^[^1]

This split respects security boundaries: Azure AI Foundry runs in Microsoft's environment, your ops controller runs where kubeconfig and cluster access live.

Prerequisites

Azure resources

- Azure subscription with permissions to create and use Microsoft Foundry / Azure AI Foundry resources (Owner/Contributor plus appropriate project-level roles).^[^2]
- Azure AI Foundry (Microsoft Foundry) project created via the portal at `ai.azure.com`.^[^3]
- At least one chat model deployment (for example, a GPT-4-class model) in the project.^[^3]

Local environment

On your development machine:

- Python 3.10+.
- `kubectl` and/or the Kubernetes Python client, configured to access your test clusters.
- Azure CLI (`az`) installed and logged in (`az login`).
- Python packages:
 - `azure-ai-projects` and `azure-identity` (for Azure AI Projects/Agents client).^[^1]
 - `fastapi` or `flask` (optional, if you expose an HTTP controller) or just plain Python for a local controller.
 - `uvicorn` (if using FastAPI) for local testing.

Step 1 – Create the Azure AI Foundry project and deploy a model

1. Sign in to Azure AI Foundry at `https://ai.azure.com` and create or open a Foundry project.^[^3]
2. In the project, go to **Models** and deploy a suitable chat completion model (the portal guides you through choosing a base model and creating a deployment).^{[4][3]}
3. Note the **model deployment name**; it is used when creating agents and tools through the SDK.^[^4]

Microsoft's quickstarts show the pattern of working inside a Foundry project endpoint and referencing model deployments by name in agent configuration and code.^{[4][3]}

Step 2 – Prototype the agent in Agents playground

Before wiring tools and code, define your agent's high-level behavior using the portal.

1. In your project, navigate to Agents → Playgrounds.^[5]
2. Click Create agent and configure:
 - **Model/deployment:** select the chat model deployment from Step 1.
 - **Name:** for example, aks-ops-agent.
 - **Instructions** (system prompt), for example:
 - "You are an Azure Kubernetes operations assistant for SRE and platform teams.">
 - "You never fabricate cluster state; you only act on data from approved tools."
 - "When a user asks for an action (e.g., list pods, describe node), you:
 1. Ask which cluster/namespace/context to use, 2) Propose a concrete command or API call, 3) Ask for explicit approval before calling any tool that executes against infrastructure."
 - Leave tools off for now; the goal is to validate conversation patterns.
3. Test a few conversations in the playground to ensure the agent asks the right clarifying questions and proposes reasonable commands.^[5]

At this stage it will still only "suggest" commands, not execute them; this is expected.

Step 3 – Choose the tool mechanism: function calling vs. OpenAPI

Foundry Agent Service supports tools via a tool catalog, including built-ins (Code Interpreter, web search, file search) and custom tools exposed via **function calling** or **OpenAPI**.^[6]

- **Function calling** is appropriate when your controller is written in the same codebase that talks to the agent: you define functions and the SDK passes structured tool calls to you.^{[6][7]}
- **OpenAPI tools** are appropriate when you expose your controller as an HTTP service and want the agent to call it like any other external API.

For a local desktop-centric workflow where your code both talks to the agent and executes `kubectl`, **function calling** is typically simpler.

The rest of this document assumes function calling, but the patterns map cleanly onto OpenAPI tools as well.

Step 4 – Define the logical tool surface

Design tools narrowly and explicitly instead of exposing raw shell access. Microsoft's guidance for tools stresses small, well-typed operations with clear contracts.^[6]

For initial Kubernetes operations, define tools such as:

- `list_pods(cluster_name, namespace, label_selector=None)`

- `describe_pod(cluster_name, namespace, pod_name)`
- `get_events(cluster_name, namespace, pod_name=None)`

Each tool:

- Takes `cluster_name` and `namespace` explicitly (no hidden context).
- Has a description explaining when it should be used.
- Returns structured data (lists/dictionaries) rather than arbitrarily long plain text where possible.

Your agent instructions should state that it must always:

- Ask the user which `cluster` and `namespace` to use if these are not already known.
- Ask for `permission` before invoking any tool that runs cluster commands.

Step 5 – Implement the local ops controller

The `ops controller` is your local process that:

- Holds Kubernetes credentials.
- Executes cluster actions when the agent requests a tool call.
- Enforces human approval and sanitizes outputs.

A minimal version can be a single Python script that:

1. Receives a function call request from the agent client.
2. Prints a prompt like: "Agent wants to run `list_pods` on cluster X / namespace Y. Approve? (y/n)".
3. If approved, runs the corresponding `kubectl` or Kubernetes Python client call.
4. Returns the result to the agent as the tool output.

More advanced versions can:

- Run as a local web service.
- Integrate with VS Code or a web UI for richer approval UX.
- Implement logging, audit, and RBAC.

Microsoft's Kubernetes + Azure AI agent examples describe similar patterns where agents discover and assess clusters by calling tools that interact with Azure resources rather than the agent process itself having direct access.

Step 6 – Set up the Azure AI Projects client locally

To move from portal to code, use the Azure AI Projects/Agents SDK. Microsoft's Python overview describes how to connect to a Foundry project using a project endpoint and `DefaultAzureCredential`.^[1]

6.1 Configure environment variables

On your desktop, set:

- `AZURE_AI_PROJECT_ENDPOINT` – your Foundry project endpoint (visible in the portal or through "View code" in Agents playground).^{[8][4]}
- `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET` if you use a service principal, or rely on `az login` for `DefaultAzureCredential`.^[^1]

The project endpoint has a form similar to:

```
https://<resource-name>.ai.azure.com/api/projects/<project-name>
```

Foundry's endpoint documentation and Q&A examples show how to retrieve this value from the portal.^{[8][4]}

6.2 Install SDKs

Install the required Python packages:

```
pip install azure-ai-projects azure-identity
```

The SDK overview documents these as the main packages for Python integration with Foundry projects and agents.^[^1]

Step 7 – Create the agent with tools via SDK

While you can create an agent in the portal, function tools are typically defined in code to give you full control over signatures and routing. Microsoft's function calling docs show how tools are described with names, descriptions, and JSON schemas.^{[7][6]}

7.1 Define tool schemas

Define a tool schema in Python that matches your planned Kubernetes actions, for example:

```
list_pods_tool = {
    "name": "list_pods",
    "description": "List pods in a given Kubernetes cluster and namespace.",
    "parameters": {
        "type": "object",
        "properties": {
            "cluster_name": {"type": "string", "description": "Logical name of the"},
            "namespace": {"type": "string", "description": "Kubernetes namespace"},
            "label_selector": {"type": "string", "description": "Optional pod label"},
        },
        "required": ["cluster_name", "namespace"],
    },
}
```

This follows the JSON schema-based pattern described in the function calling documentation for tools.^{[6][7]}

7.2 Create the agent in code

Using the Azure AI Projects client, create an agent that references your model deployment and registers tools:

```
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

project = AIProjectClient(
    endpoint=os.environ["AZURE_AI_PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
)

agent = project.agents.create_agent(
    model="<your-model-deployment-name>",
    name="aks-ops-agent",
    instructions=(
        "You are an Azure Kubernetes operations assistant for SRE and platform teams"
        "You never fabricate cluster state; you only act on data from approved tools."
        "When a user asks for an action that affects infrastructure, you must first ask"
        "cluster and namespace to use, then ask for explicit human approval before c"
    ),
    tools=[list_pods_tool],
)
```

The exact method names and parameters may vary slightly with SDK versions; Microsoft's agent service quickstart shows the canonical pattern for creating agents with tools using this client.^[^3]

Step 8 – Implement the approval loop and tool execution

With the agent defined, the next step is to run a conversation where the agent can emit tool calls and your controller decides whether to execute them.

8.1 Conversation primitives

Foundry agents are used via **threads**, **messages**, and **runs**, as shown in Microsoft's quickstarts:

- **Thread:** conversation context for a particular user/session.
- **Message:** user or agent message attached to a thread.
- **Run:** execution of the agent on a thread, which may involve tool calls.

The SDK exposes operations to create threads, add messages, start runs, and read back messages and tool calls.^[^3]

8.2 Detecting tool calls

When the agent decides to use a tool, the run result includes **tool invocation payloads** (structured function calls or tool instructions) described in the function calling documentation.^{[7][6]}

Your code must:

- Inspect the run result for tool call requests.
- For each tool call, surface the details to the user for approval.

8.3 Sample loop with approval

A conceptual loop (simplified for clarity) looks like this:

```
thread = project.agents.create_thread()

project.agents.create_message(
    thread_id=thread.id,
    role="user",
    content="List all pods in my production cluster",
)

run = project.agents.create_run(thread_id=thread.id, agent_id=agent.id)

while True:
    run = project.agents.get_run(thread_id=thread.id, run_id=run.id)

    if run.status in ("completed", "failed", "cancelled"):
        break

    if run.status == "requires_action":
        tool_calls = run.required_action.get("tool_calls", [])
        for call in tool_calls:
            if call["name"] == "list_pods":
                args = call["arguments"]
                # 1) Ask human for approval
                print(f"Agent wants to list pods on cluster={args['cluster_name']} ns=
                approve = input("Approve? (y/n): ").strip().lower() == "y"
                if not approve:
                    # Send a tool result indicating denial
                    project.agents.submit_tool_outputs(
                        thread_id=thread.id,
                        run_id=run.id,
                        outputs=[{
                            "tool_call_id": call["id"],
                            "output": {"approved": False, "reason": "User denied"},
                        }],
                    )
                    continue

                # 2) Execute real command
                pods = list_pods_in_cluster(
                    cluster_name=args["cluster_name"],
```

```

        namespace=args["namespace"],
        label_selector=args.get("label_selector"),
    )

    # 3) Send tool output back to agent
    project.agents.submit_tool_outputs(
        thread_id=thread.id,
        run_id=run.id,
        outputs=[{
            "tool_call_id": call["id"],
            "output": {"approved": True, "pods": pods},
        }],
    )

    # After submitting tool outputs, the agent continues the run
    continue

```

Exact field names (`required_action`, `tool_calls`, `submit_tool_outputs`) follow the function calling and agent API documentation for Foundry; they may differ slightly by SDK version, but the interaction pattern is the same: agent run requests tools, client submits tool outputs, run continues.^{[6][7]}

8.4 Implementing `list_pods_in_cluster`

The `list_pods_in_cluster` helper can be implemented with either:

- `kubectl` subprocess calls, or
- The Kubernetes Python client using the cluster's API server.

For example, with the Kubernetes client:

```

from kubernetes import client, config

# Load kubeconfig or in-cluster config earlier in your program
config.load_kube_config() # or load_incluster_config()

def list_pods_in_cluster(cluster_name: str, namespace: str, label_selector: str | None) -> list[dict]:
    """
    # In a multi-cluster environment you would switch contexts or use per-cluster config
    """
    v1 = client.CoreV1Api()
    pod_list = v1.list_namespaced_pod(namespace=namespace, label_selector=label_selector)
    return [
        {
            "name": pod.metadata.name,
            "phase": pod.status.phase,
            "node": pod.spec.node_name,
        }
        for pod in pod_list.items
    ]

```

The agent receives a clean, structured list of pods and can summarize or reason over it, rather than parsing raw CLI text.

Step 9 – Streaming results back to the user

To approximate a "live" experience where command results stream as they are produced, combine:

- Streaming from your ops controller to your UI, and
- Streaming chat responses from the agent to your UI.

Foundry's Responses API and agent APIs support streaming token output to clients, allowing you to incrementally display the agent's answer as it composes it.

On the controller side, for long-running operations (for example, tailing logs), you can:

- Stream partial tool outputs back to the user UI directly.
- Only send a summarized final result back to the agent as the tool output.

This keeps the agent reasoning on summarized data while still giving you real-time operational visibility.

Step 10 – Hardening and production considerations

Before trusting such an agent in serious environments, consider:

- **Scope limitation**
 - Restrict tools to read-only commands initially (`list_pods`, `describe_*`, `get_events`).
 - Add mutating operations (`delete_pod`, `scale_deployment`) only after approvals and guardrails are thoroughly tested.
- **RBAC and identity**
 - Use distinct Kubernetes credentials for the controller with least privilege, not your personal admin context.
 - For AKS, consider managed identities and Azure RBAC for the API server.
- **Audit and logging**
 - Log every tool call request, human approval/denial, command executed, and returned results.
 - Store logs centrally (for example, Azure Monitor or Log Analytics).
- **Multi-cluster design**
 - Maintain a registry mapping logical `cluster_name` values to actual contexts or API servers.
 - Enforce per-cluster policy (for example, read-only tools against production).
- **Failure modes**
 - Handle tool errors (cluster unreachable, auth failure) explicitly and return clear error messages to the agent.
 - Ensure the agent instructions tell it to surface errors without guessing at missing data.

Microsoft's tool best practice documentation emphasizes clear contracts, bounded side effects, and robust error handling when connecting agents to external systems.^[^6]

Step 11 – Optional: using OpenAPI instead of function tools

If you prefer to run the ops controller as a separate HTTP service, use an OpenAPI-based tool:

1. Implement a FastAPI/Express service with endpoints such as `POST /list_pods`.
2. Generate or hand-write an OpenAPI spec describing this API.
3. Register the OpenAPI spec as a tool in your agent configuration using Foundry's OpenAPI tool support.
4. In your controller, enforce the same approval workflow before executing cluster operations.

OpenAPI tools are particularly useful if multiple agents or services will share the same operations API.

Step 12 – End-to-end test plan

To validate that the full loop works as intended:

1. Unit test the controller functions

- Verify `list_pods_in_cluster` returns correct data for test clusters.

2. Simulate tool calls without the agent

- Manually call your approval logic and ensure approvals and denials behave as expected.

3. Dry-run the agent with tools

- Start a thread and ask: "List all pods in cluster `dev-aks` namespace `default`."
- Confirm that:
 - The agent asks for clarification if necessary.
 - The run transitions to a state where tool calls are requested.
 - Your controller prompts you for approval.
 - On approval, real `kubectl`/API calls run and results go back to the agent.
 - The agent's final message summarizes actual cluster state.

4. Negative paths

- Deny a tool call and confirm the agent responds appropriately (for example, "I cannot perform that operation without approval").
- Break cluster connectivity and confirm errors are surfaced rather than hidden.

5. Security review

- Check that no tool allows arbitrary shell command execution or unrestricted cluster mutation.

- Validate that logs are sufficient for audit.

With these steps complete, you will have an ops agent that:

- Lives in Azure AI Foundry.
- Proposes actions and asks for your approval.
- Executes real commands against your Kubernetes clusters only when approved.
- Returns structured results to support further reasoning and troubleshooting.

References

1. [I want to create a custom agent for my vs code because I don't have access to use the GitHub co pilot but I have complete access to ai foundry in azure.](#)
I need an agent that behaves like cursor, Gemini anti gravity, GitHub co pilot in vscode, lovable that supports complete full stack development not just one off chat response but in an interactive way - Yes — you can build a real Copilot/Cursor-style coding assistant around Azure AI Foundry, but not as...
2. [Generate step by step instructions on azure on how to create this agent ?](#) - Here is a concrete, Azure-native, step-by-step path to create the agent in Foundry with Code Interpr...
3. [Quickstart - Create a new Foundry Agent Service project \(classic\)learn.microsoft.com › en-us › azure › foundry-classic › agents › quickstart](#) - Use this guide to start using Foundry Agent Service. (classic)
4. [Quickstart - Create a new Foundry Agent Service project](#) - Use this guide to start using Foundry Agent Service.
5. [Getting Started - Build your code-first agent with Azure AI Foundry](#)
6. [Build An AI Agent From Scratch Using Microsoft Foundry](#) - Join the Free Azure Innovation Station Community!
<https://azureinnovationstation.com/community>

Azur...

7. [Build without boundaries](#) - Azure AI Foundry
8. [Quick Start - Azure AI Foundry Workshop](#) - A practical 4-5 hour workshop on building intelligent health and dietary advice agents with Azure AI...